JAVA 程序设计

授课人：何毅
办公室：1909

第16讲 内容向导

16.1 字符流类

16.2 文件处理

16.1 字符输入输出流

1. 字符输入输出

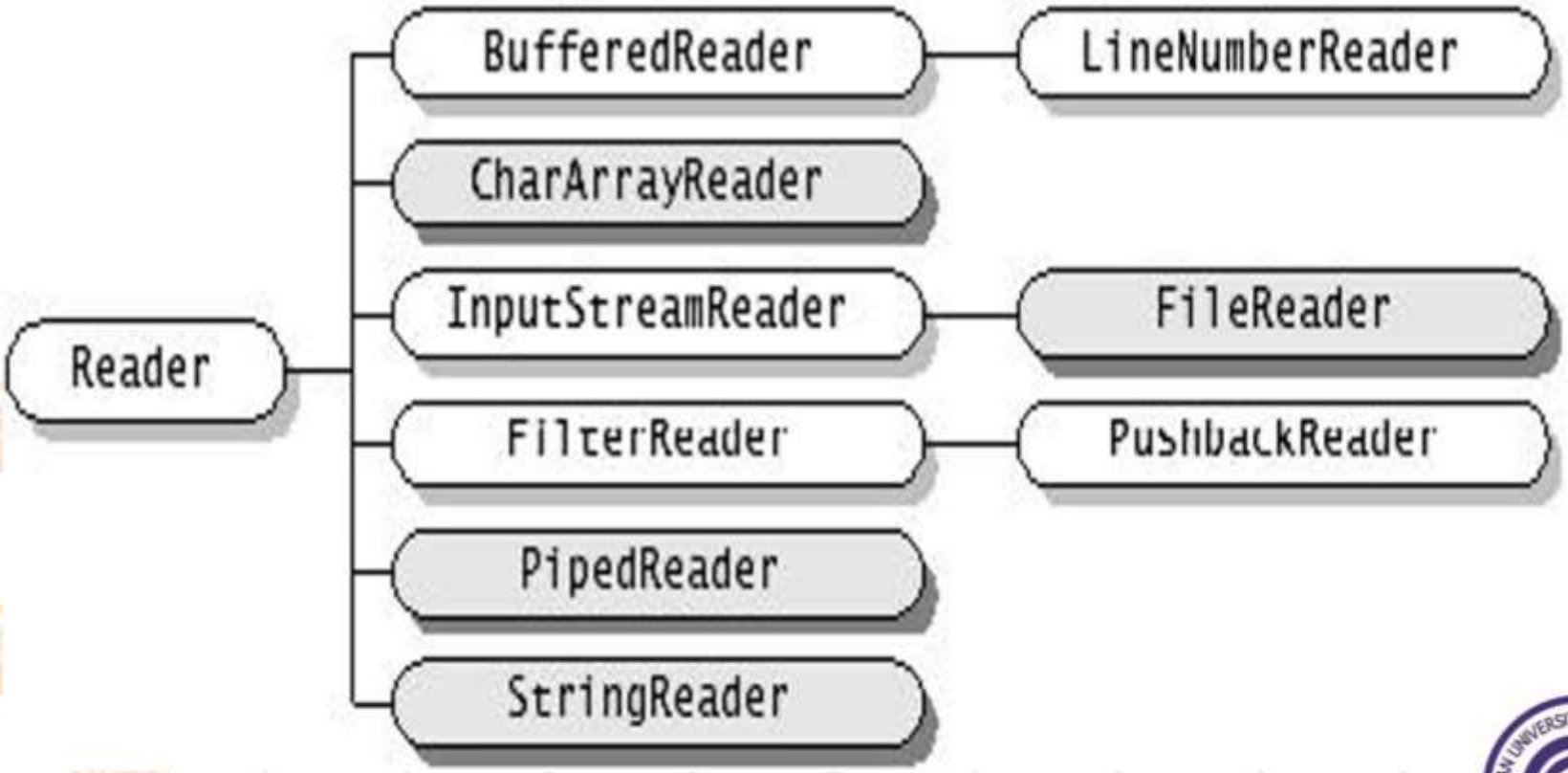
- 字符流一次读写16位二进制数，并将其作为一个字符而不是二进制位来处理。

- Reader和Writer提供了基于字符的IO（character-based）

- 基于字节的流无法正确处理基于16位的unicode 字符，经常出现乱码，Reader和Writer在所有IO操作上提供对unicode 的支持。

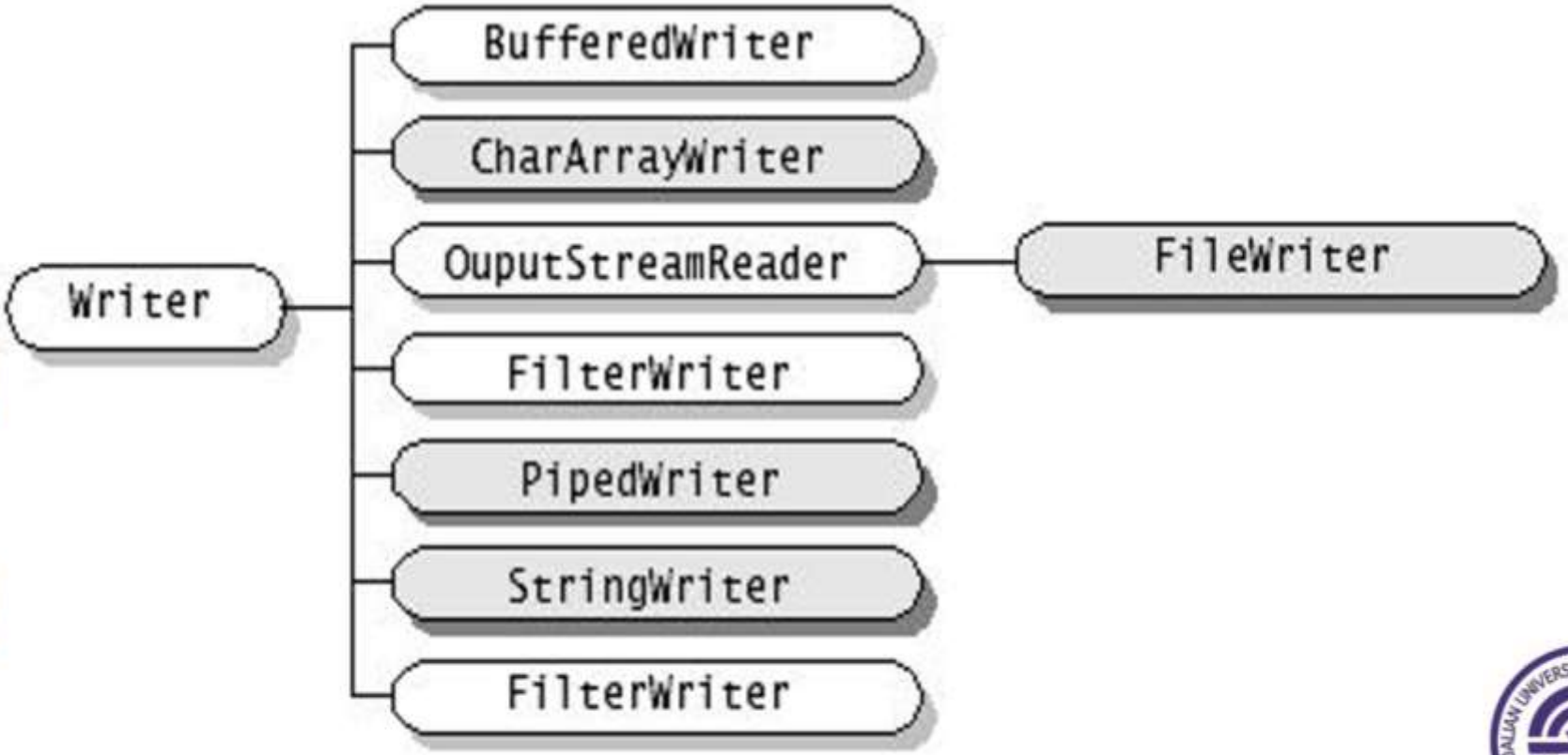
➢java.io包中的类

1）字符流Reader类



➢java.io包中的类

2）字符流Writer类



2.文件字符输入输出流

FileReader和**FileWriter**用于文本文件的输入输出处理，与文件字节输入流**FileInputStream**和文件字节输出流**FileOutputStream**的功能相似，但处理的基本单位是字符。

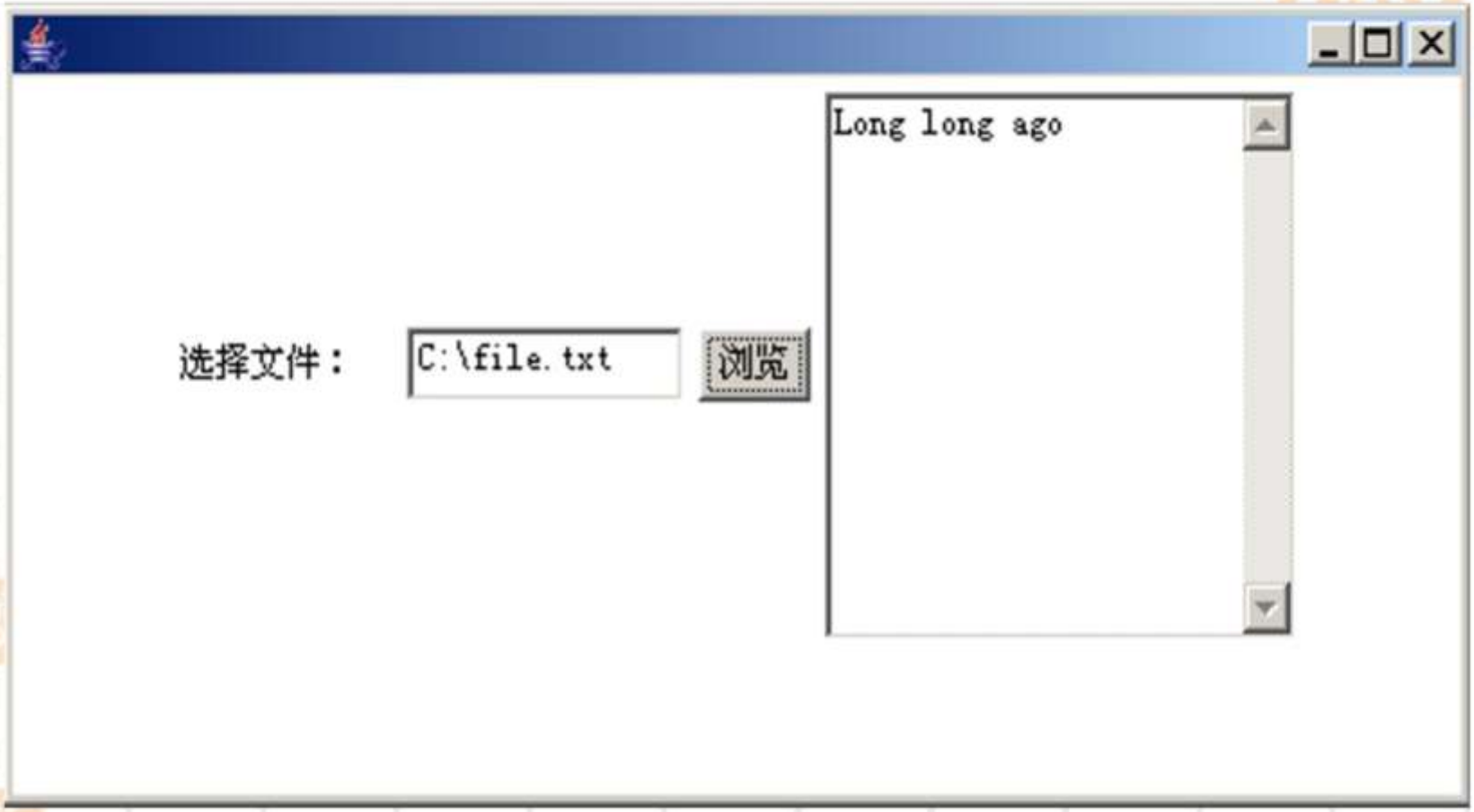
3.字符缓冲流

1.BufferedReader缓冲字符输入流

- public BufferedReader(Reader in)
- public int read() throws IOException
- public int read(char[] cbuf, int off, int len) throws IOException
- public String readLine() throws IOException

2.BufferedWriter缓冲字符输入流

- public BufferedWriter(Writer out)
- public void flush() throws IOException
- public void newLine() throws IOException
- public void write(char[] cbuf, int off, int len) throws IOException
- public void write(String s, int off, int len) throws IOException
- public void write(int c) throws IOException



```
import java.awt.*;  import java.awt.event.*;
import java.io.*;   import javax.swing.*;
public class test extends JFrame implements ActionListener{
   FileDialog m_fd;
   JTextField m_txtFilename=new JTextField(10);
   JLabel m_lblChoosefile=new JLabel("选择文件：");
   JButton m_btnChoose=new JButton("浏览");
   JTextArea m_txtareaFilecontent=new JTextArea(10,20);
   String m_sTemp ,inFilename,outFilename;
   StringBuffer m_buf=new StringBuffer();
    test(){
      setDefaultCloseOperation(EXIT_ON_CLOSE);
      this.setLayout(new FlowLayout());
      add(m_lblChoosefile);
      add(m_txtFilename);
      add(m_btnChoose);
      add(m_txtareaFilecontent);
      m_btnChoose.addActionListener(this);
      setSize(500,250);
      setVisible(true);
}
```

```
public void actionPerformed(ActionEvent e){
String cmd=e.getActionCommand();
    if(cmd=="浏览"){
      m_fd=new FileDialog(this,"打开文件",FileDialog.LOAD);
        m_fd.setDirectory("c:\\");m_fd.setVisible(true);
       inFilename=m_fd.getDirectory()+m_fd.getFile();
       m_txtFilename.setText(inFilename);
       try{
          BufferedReader br=new BufferedReader(new
                                FileReader(inFilename));
          outFilename=m_fd.getDirectory()+"b.txt";
          BufferedWriter bw=new BufferedWriter(new
                                FileWriter(outFilename));
          String s;
          while((s=br.readLine())!=null){
           m_buf.append(s);
       }
      bw.write(m_buf.toString());
      br.close();   bw.close();
      m_txtareaFilecontent.setText(m_buf.toString());
       }catch(Exception ee){ } }}
  public static void main(String[] args){ new test(); }}
```

16.2 文件处理

1. 文件

- 文件是存储在外部存储介质上的具有标识名的一组相关信息的集合。
- 文件系统性提供“按名存取”实现文件的存储和检索。
- 树形目录结构
- 文件分为文本文件和二进制文件。
- 文件的存取方法有顺序存取、随机存取和索引存取。

- **File类**

  主要用于描述系统中的文件在磁盘上的存储情况，而**File**类的对象主要用来获取文件本身的一些信息，还可利用**File**对象来对文件系统做一些查询与设置的动作，但不涉及对文件的读写操作。

- 不管是文件还是目录，在**Java**中都以**File**的实例来表示。

- File类专门提供一个抽象，用于以机器无关的方式处理大多数机器有关的、复杂的文件和路径名问题。文件名是一个字符串。File类是文件名的一个包装类。

- 使用下面的语句可以创建一个File对象：

```
File file = new File("temp.txt");
```

- 使用文件名和路径时，总是假定所使用的是主机的文件名规则。因此，在windows下的路径分隔符是\ ，unix下是/。

2.构造方法

- public File(String pathname)
- public File(String parent, String child)
- public File(File parent, String child)
- public File(URI uri)

3.成员方法

- public boolean canRead()
- public boolean createNewFile() throws IOException
- public boolean delete()
- public boolean exists()
- public String getAbsolutePath()
- public String getName()
- public boolean isDirectory()
- public long length()
- public String[] list()
- public boolean mkdir()

- **File类中的方法**
  - public String[] list(FilenameFilter filter)
  - public File[] listFiles(FileFilter filter)
  - public File[] listFiles(FilenameFilter filter)
- FilenameFilter 接口
  - boolean accept(File dir, String name)
- FileFilter 接口
  - boolean accept(File pathname)

4.File类的主要方法

exists()：判断文件是否存在

getName()：获得文件名称，返回字符串

getPath()：获得文件的完整路径，返回字符串

getParent()：获得父路径，返回字符串

createNewFile()：创建文件，返回布尔值

➢注意：创建一个File对象时，如果它代表的文件不存在，系统不会自动创建，必须要调用createNewFile()来创建。

5. 文件流（File Streams）的创建

- 必须使用文件数据流对磁盘进行读写。 FileInputStream和FileOutputStream用于字节流，FileReader和 FileWriter用于字符流。

- 创建文件数据流的构造方法：

```
public FileInputStream(String filename)
public FileInputStream(File file)
public FileOutputStream(String filename)
public FileOutputStream(File file)
public FileReader(String filename)
public FileReader(File file)
public FileWriter(String filename)
public FileWriter(File file)
```

6.文件流的创建

➢创建FileInputStream/FileOutputStream对象或者创建FileReader/FileWriter对象对文件进行读写操作：

```
FileInputStream infile = new
                         FileInputStream("in.dat");
FileOutputStream outfile = new
                         FileOutputStream("out.dat");
FileReader infile = new FileReader("in.dat");
FileWriter outfile = new FileWriter("out.dat");
```

➢在FileInputStream中实现了InputStream中象read()之类的抽象方法，在FileOutputStream中实现了OutputStream中的抽象方法write(int b)。

```
import java.io.*;
public class CopyFileUsingByteStream{
   public static void main(String[] args){
    FileInputStream fis = null;
    FileOutputStream fos = null;
    try{
      fis = new FileInputStream(new File("FInputFile.txt"));
      File file = new File("FOutputFile.txt");
      if (file.exists()) {
        System.out.println("file  already exists");
        return;
      }
      else
        fos = new FileOutputStream("FOutputFile.txt");
        System.out.println("The file FInputFile " +  + " has "+
        fis.available() + " bytes");
```

```
int r;
        while ((r = fis.read()) != -1) {
        fos.write(r);
      } }
    catch (FileNotFoundException ex) {
      System.out.println("File FInputFile not found: " +);
    }
    catch (IOException ex) {
      System.out.println(ex.getMessage());
    }
    finally {
      try {
       if (fis != null) fis.close();
        if (fos != null) fos.close();
      }
      catch (IOException ex){
        System.out.println(ex);
      } } }}
```

7. 实例

声明窗口类ReadFile，在窗口中输入用户名和密码，并将用户名和密码保存到文件Secret.txt中。

```
import java.awt.*;
import java.awt.event.*;
import java.io.*; import javax.swing.*;
public class ReadFile extends JFrame implements ActionListener{
    JLabel m_id=new JLabel("用户名：");
    JLabel m_key=new JLabel("密码：");
    JButton m_input=new JButton("存入");
    JTextField Id=new JTextField(10);
    JPasswordField Keyid=new JPasswordField(10);
    String outFilename;
    StringBuffer m_buf=new StringBuffer();
    ReadFile(){
     setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setLayout(new FlowLayout());
        add(m_id);     add(Id);
        add(m_key);    add(Keyid);
        add(m_input);
        m_input.addActionListener(this);
        setSize(400,100);
        setVisible(true);  }
```

```
public void actionPerformed(ActionEvent e){
      String cmd=e.getActionCommand();
      if(cmd=="存入"){
         String Idstring;
         String Keyidstring;
         Idstring = Id.getText();
         Keyidstring = Keyid.getText();
         m_buf.append("用户名是 ");
         m_buf.append(Idstring);
         m_buf.append("  密码是 ");
         m_buf.append(Keyidstring);
  try{
  outFilename="c:\\"+"Secret.txt";
  BufferedWriter bo=new BufferedWriter(new FileWriter(outFilename));
  bo.write(m_buf.toString());
  bo.close();
     }catch(Exception ee){ }  }}
         public static void main(String[] args){ new ReadFile(); }
}
```

复习与预习

复习：

1. 流及文件

预习：

数据库编程